

REPUBLIC OF CAMEROON
Peace-Work-Fatherland

MINISTRY OF HIGHER EDUCATION

UNIVERSITY OF EBOLOWA

REPUBLIQUE DU CAMEROUN
Paix-Travail-Patrie

MINISTRY OF HIGHER EDUCATION

UNIVERSITY INSTITUTE OF THE TROPICS



DESIGN AND DEVELOPMENT OF AN INTERNSHIP MANAGEMENT SYSTEM.

CASE STUDY : CAMEROON

**RESEARCH REPORT SUBMITTED IN PARTIAL FULFILMENT FOR THE AWARD
OF PROFESSIONAL BACHELOR OF WEB DIGITAL AND INFORMATION
TECHNOLOGIES DEGREE OF UNIVERSITY OF EDOLOWA
FIELD : TECHNOLOGY**

SPECIALTY: WEB DIGITAL and INFORMATION TECHNOLOGY

Presented by:

KAMGUEM WAMBO YORICK NAZERL

Matricule Number :

Under The Supervision Of :

PROFESSIONAL SUPERVISOR

Mr. FOKO GABIN

PROFESSIONAL SUPERVISOR

Mr. PEUPIE GUY BERTRAND

ACADEMIC YEAR 2025-2026

DECLARATION

I, KAMGUEM WAMBO YORICK , declare that this work presented is original and has not been published or submitted elsewhere for the same or other purposes. All sources used have been duly acknowledged.

Signature:_____

Date: _____

CERTIFICATION

This is to certify that this research report, titled "DESIGN AND DEVELOPMENT OF AN INTERNSHIP MANAGEMENT SYSTEM: INTERNLINK," has been prepared by KAMGUEM WAMBO YORICK under my supervision and is submitted in partial fulfillment of the requirements for the award of Professional Bachelors of WEB DIGITAL and INFORMATION TECHNOLOGY Degree of University of Yaoundé II.

Academic Supervisor: _____

Date: _____

Professional Supervisor: _____

Date: _____

DEDICATION

This work is dedicated to my lovely family who since I'm born , do the maximum to permit me be in good health and emotionally stable.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to all my friends who help me accomplish my work.

LIST OF FIGURES

Figure 1:Agile model.....	26
Figure 2:Use case diagram.....	41
Figure 3:Class diagram.....	42
Figure 4:Sequence diagram (students application).....	43
Figure 5:Sequence diagram (weekly repport).....	44
Figure 6:Deployment diagram.....	45

LIST OF TABLES

LIST OF ABBREVIATIONS/ACRONYMS

- SPA: Single-Page Application
- JWT: JSON Web Token
- ORM: Object-Relational Mapping (Prisma)
- API: Application Programming Interface
- i18n: Internationalization
- RBAC: Role-Based Access Control
- OTP: One-Time Password
- MDX: Markdown with JSX (Report Guide Editor)
- CRUD: Create, Read, Update, Delete
- CSS: Cascading Style Sheets (Tailwind CSS v4)
- JS: JavaScript (Next.js / TypeScript)
- UAT: User Acceptance Testing

ABSTRACT

This report details the design and development of InternLink, a full-stack internship management platform tailored specifically for Cameroon. InternLink bridges the gap between education and employment by connecting university students with verified companies. The platform manages the entire internship lifecycle from offer discovery and application to weekly report submission, supervisor validation, and digital certificate issuance. InternLink provides a centralized, transparent, and efficient solution to the challenges of internship management in Cameroon, fostering a more dynamic and accessible environment for both students and employers.

TABLE OF CONTENT

DECLARATION.....	i
CERTIFICATION.....	ii
DEDICATION.....	iii
ACKNOWLEDGMENTS.....	iv
LIST OF FIGURES.....	v
LIST OF TABLES.....	vi
LIST OF ABBREVIATIONS/ACRONYMS.....	vii
ABSTRACT.....	viii
CHAPTER ONE : INTRODUCTION.....	1
1.0 INTRODUCTION.....	1
1.1 BACKGROUND OF THE STUDY.....	1
1.2 PROBLEM STATEMENT.....	3
1.3 RESEARCH QUESTIONS.....	4
1.4 RESEARCH OBJECTIVES.....	4
1.5 RESEARCH HYPOTHESIS.....	4
1.6 SIGNIFICATION OF THE STUDY.....	5
1.7 SCOPE OF THE STUDY.....	5
1.8 ORGANISATION OF THE STUDY.....	6
CHAPTER TWO: LITERATURE REVIEW.....	7
2.0 INTRODUCTION.....	7
2.1 CONCEPTUAL REVIEW.....	7
2.1.1 INTERNSHIP MANAGEMENT SYSTEM (IMS).....	7
2.1.2. THE DEPENDENT VARIABLE : STUDENT SATISFACTION.....	9
2.1.3 FULL-STACK DEVELOPMENT.....	12
2.1.4 EXPERIENTIAL LEARNING AND INTERSNHIPS.....	12
2.2 THEMATIC REVIEW.....	12
2.2.1 TRENDS IN INTERNHIP MANAGEMENT.....	12
2.2.2 SOFTWARE DEVELOPMENT METHODOLOGIES.....	13
2.3 EMPERICAL REVIEW.....	13
2.3.1 EXISTING INTERNSHIP MANAGEMENT SYSTEMS.....	13

2.3.2 CHALLENGES IN INTERNSHIP PLACEMENT IN DEVOLOPING COUNTRIES (FOCUS: CAMEROON).....	13
2.4 MOBILE TECHNOLOGIES.....	14
Advantages & Limitations of VS Code.....	15
2.4.1 TECHNICAL STACK.....	16
2.5 SDLC MODEL.....	25
2.5.1 IMPORTANCE OF SDLC.....	25
2.5.2 Agile Model.....	26
Advantages & Disadvantages of the Agile Model.....	28
CHAPTER THREE: METHODOLOGY.....	34
3.1 STUDY DESIGN.....	34
3.2. STUDY POPULATIONS AND SAMPLING TECHNIQUES.....	36
3.2.1. TARGET POPULATION.....	36
3.2.2. SAMPLE SIZE AND SAMPLING TECHNIQUE.....	36
3.3. DATA COLLECTION PROCEDURE.....	37
3.4. DATA ANALYSIS PROCEDURE AND ESTIMATION TECHNIQUES.....	38
3.5. ETHICAL CONSIDERATION.....	39
3.6 STUDY AND PROJECT DURATION.....	40

CHAPTER ONE : INTRODUCTION

1.0 INTRODUCTION

Internships represent a critical bridge between academic learning and professional practice, offering students invaluable opportunities to apply theoretical knowledge, develop practical skills, and gain exposure to the professional world. In an increasingly competitive global job market, the quality and effectiveness of internship programs significantly influence a student's career trajectory and overall satisfaction with their educational experience. However, the administration and management of these programs can be complex, often involving numerous stakeholders including students, academic institutions, and host organizations. This complexity frequently leads to inefficiencies, communication breakdowns, and a suboptimal experience for all parties involved. This study delves into the design and development of an Internship Management System (IMS) as a strategic solution to streamline these processes, enhance operational efficiency, and ultimately improve student satisfaction with their internship experiences.

1.1 BACKGROUND OF THE STUDY

The landscape of higher education is continuously evolving, with a growing emphasis on practical, experiential learning to prepare students for the demands of the modern workforce. Internships have emerged as a cornerstone of this pedagogical shift, providing a structured environment for students to acquire hands-on experience, cultivate professional networks, and refine their career aspirations. The National Association of Colleges and Employers (NACE) defines an internship as a form of experiential learning that integrates knowledge and theory learned in the classroom with practical application and skill development in a professional setting. This definition underscores the dual benefit of internships: for students, it is a period of significant personal and professional growth, and for institutions, it is a vital component of their commitment to producing work-ready graduates.

Globally, the demand for well-managed internship programs has led to the adoption of various technological solutions. These systems aim to centralize information, automate administrative tasks, and facilitate seamless communication among students, faculty, and employers. The benefits of such systems are manifold, including improved matching of students to suitable

internships, enhanced tracking of student progress, and more efficient reporting mechanisms. For instance, studies on the usability of Internship Management Systems (IMS) highlight their potential to reduce administrative burdens and increase the overall effectiveness of the internship experience by digitizing processes that were traditionally manual and prone to error.

However, the implementation and effectiveness of these systems are not uniform across all regions, particularly in developing countries. In Cameroon, like many other African nations, higher education institutions face unique challenges in managing internship programs. Research indicates that institutions often struggle with the organization of research and student internships, leading to a lack of centralized management and oversight. This often results in a fragmented approach, where information is scattered across various platforms, communication is inconsistent, and the monitoring of student progress is inadequate. A case study involving Viettel Cameroon, for example, revealed significant challenges in managing internships, including large archives, a lack of traceability and monitoring, and the archaic reliance on tools like MS Excel for critical administrative tasks. Such inefficiencies not only hinder the smooth operation of internship programs but also negatively impact the quality of the experience for students.

The broader context of higher education in Cameroon also presents its own set of complexities. The dual system of education, while offering diverse options, can also lead to disparities in quality and access to resources . Furthermore, the adoption of information technology in Cameroonian universities, while progressing, still faces hurdles related to infrastructure, funding, and technical expertise . These challenges underscore the urgent need for robust and tailored technological solutions that can address the specific needs of internship management within the Cameroonian educational landscape. The development of an effective platform like InternLink, therefore, is not merely about digitizing existing processes but about transforming the entire internship ecosystem to better serve students and institutions.

In this context, the focus on student satisfaction becomes paramount. A positive internship experience, facilitated by an efficient management system, can significantly enhance a student's perception of their academic program and institution. Conversely, a poorly managed internship can lead to frustration, disengagement, and a diminished sense of value from their educational investment. Therefore, designing an IMS that prioritizes user experience and addresses the pain points of students is crucial for fostering higher levels of satisfaction and ensuring the successful integration of practical learning into academic curricula.

Traditional methods of internship search and management, such as manual applications, reliance on personal networks, and paper-based reporting, are time-consuming and prone to administrative bottlenecks. Students often struggle to find relevant and verified opportunities, while companies face hurdles in effectively managing the application process, tracking intern progress, and ensuring compliance with educational requirements. Furthermore, the validation of internship activities and the issuance of certificates can be cumbersome, adding to the administrative burden for all parties involved. The absence of a centralized, efficient, and transparent system exacerbates these issues, hindering the seamless integration of academic learning with practical industry experience.

It is within this context that InternLink emerges. InternLink is a full-stack internship management platform tailored specifically for Cameroon. Its mission is to bridge the gap between education and employment by connecting university students with verified companies offering internship and apprenticeship opportunities. By leveraging modern web technologies, InternLink creates a robust ecosystem that facilitates the entire internship lifecycle from offer discovery and application to weekly report submission, supervisor validation, and certificate issuance all managed within a single, cohesive platform.

1.2 PROBLEM STATEMENT

The current manual or semi-manual approaches to internship management in many higher education institutions, particularly in regions like Yaounde Cameroon, are fraught with inefficiencies and limitations. The absence of a dedicated, integrated platform like InternLink intensify these issues. These traditional methods often involve extensive paperwork, disparate communication channels, and a lack of centralized data management, leading to significant administrative burdens for faculty, staff, and students alike. The absence of a unified system results in difficulties in tracking internship placements, monitoring student progress, and evaluating program effectiveness. Students frequently encounter challenges in finding relevant internship opportunities, submitting required documentation, and receiving timely feedback, which collectively detracts from the overall quality of their experiential learning. Furthermore, the lack of transparent and accessible information often leads to confusion and dissatisfaction among students regarding the internship process. This fragmented approach not only compromises the administrative efficiency of institutions but also directly impacts student satisfaction, a critical indicator of educational quality and program success. Without a well-

structured and technologically supported management system, the potential for internships to significantly enhance student employability and professional development remains largely unfulfilled.

1.3 RESEARCH QUESTIONS

Main Research Question: How can the design and development of InternLink, a specialized Internship Management System (IMS), effectively enhance student satisfaction with internship programs in higher education institutions in Cameroon?

Specific Research Questions:

- 1.What are the key challenges faced by students, academic staff, and host organizations in managing internships in higher education institutions in Cameroon?
- 2.How can the features and functionalities of InternLink be designed to address these identified challenges and improve the efficiency of internship administration?
- 3.To what extent does the implementation of InternLink influence student satisfaction with their internship experiences in terms of perceived usefulness and ease of use?

1.4 RESEARCH OBJECTIVES

Main Research Objective: To design and develop InternLink, a specialized Internship Management System (IMS), aimed at significantly enhancing student satisfaction with internship programs in higher education institutions in Cameroon.

Specific Research Objectives:

- 1.To identify and analyze the existing challenges in internship management from the perspectives of students, academic staff, and host organizations in Cameroon.
- 2.To propose and implement a set of features and functionalities for InternLink that effectively addresses the identified challenges and streamlines internship administration.
- 3.To evaluate the impact of the developed InternLink on student satisfaction, specifically assessing its perceived usefulness and ease of use among interns.

1.5 RESEARCH HYPOTHESIS

- **H01:** The implementation of a centralized digital platform has no significant effect on the accessibility and transparency of internship opportunities for students.

- **H02:** Real-time digital monitoring and structured feedback mechanisms do not significantly improve the efficiency of internship supervision and the certification process.

1.6 SIGNIFICATION OF THE STUDY

The development of InternLink holds significant implications for various stakeholders within Cameroon's educational and professional sectors. For university students, the platform provides unprecedented access to verified opportunities, reducing search effort and ensuring equitable access. The streamlined reporting and validation processes enhance their learning experience and ensure timely recognition through digital certificates.

For companies and organizations, InternLink offers an efficient mechanism for recruiting talent. The platform simplifies application management, allows for structured intern supervision via a dedicated Supervisor role, and offers subscription plans (Starter, Scholar, Pro) payable via local mobile money solutions. This reduces administrative overhead and improves the quality of the internship experience.

Educational institutions benefit from improved oversight of student placements. InternLink provides tools for tracking progress and validating reports, ensuring internships meet academic standards. The platform's Report Guide feature, managed by admins, provides a standardized framework for academic writing and professional reporting.

More broadly, InternLink contributes to national development by bridging the gap between education and employment. It supports the development of a skilled workforce, reduces youth unemployment, and demonstrates how modern technologies like Next.js, Socket.io, and WhatsApp API can solve local administrative challenges in Cameroon.

1.7 SCOPE OF THE STUDY

This study focuses on the design and development of InternLink, a specialized Internship Management System (IMS), and its impact on student satisfaction. The thematic scope is specifically limited to the functionalities and user experience of InternLink, with student satisfaction being measured primarily through perceived usefulness and perceived ease of use, drawing from the Technology Acceptance Model. The geographical scope of the study is confined to higher education institutions located in Yaounde, Cameroon. The insights and data

collected will be specific to this region, acknowledging that challenges and technological adoption rates may vary elsewhere. The time scope of the study encompasses a 10-week internship period, during which the system's development and initial evaluation will be conducted, focusing on the experiences of students undertaking internships within this timeframe.

1.8 ORGANISATION OF THE STUDY

This report is structured into five main chapters.

Chapter One: Introduction, provides the background, problem statement, objectives, and scope.

Chapter Two: Literature Review, explores the theoretical foundation, modern web technologies like Next.js, and the internship landscape.

Chapter Three: Methodology, details the technical architecture, including the 13 database models, 22 API route groups, and the Next.js SPA structure.

Chapter Four: System Implementation and Testing, describes the development of the 4 user roles, the design system, and the integration of real-time features. Finally,

Chapter Five: Conclusion and Recommendations, summarizes the findings and proposes future enhancements such as AI matching and native mobile apps.

CHAPTER TWO: LITERATURE REVIEW

2.0 INTRODUCTION

This chapter provides a comprehensive review of existing literature pertinent to the design and development of an Internship Management System (IMS) and its impact on student satisfaction. It delves into the theoretical underpinnings of internship programs, the technological aspects of management systems, and the psychological factors influencing student satisfaction. The review is structured into conceptual, thematic, and empirical sections, culminating in a presentation of the internship activities that inform the practical context of this study. By synthesizing current knowledge, this chapter aims to establish a robust theoretical framework and identify research gaps that this study seeks to address.

2.1 CONCEPTUAL REVIEW

2.1.1 INTERNSHIP MANAGEMENT SYSTEM (IMS)

An **Internship Management System (IMS)** is a software application designed to streamline and automate the various processes involved in managing internships. These systems typically serve as a central hub for students, educational institutions, and employers, facilitating communication and administrative tasks throughout the internship lifecycle. According to Mydyti and Kadriu (2020) [1], an IMS is a fundamental fragment of communication between internship seekers, employers, and educational institutions, playing a crucial role in improving the digitization of processes related to internships. Key functionalities often include internship posting and discovery, application submission and tracking, reporting and evaluation, and certificate issuance [2]. The primary goal of an IMS is to enhance efficiency, transparency, and effectiveness in connecting students with relevant opportunities and managing their practical training experiences.

Importance of an Internship Management System

The importance of InternLink stems from its capacity to address critical challenges inherent in traditional internship management and to unlock significant benefits for all involved parties:

1.Enhanced Efficiency and Automation: InternLink automates repetitive administrative tasks, such as application processing, document management, and communication workflows. This significantly reduces the time and effort spent by academic staff and company HR, allowing them

to focus on more strategic aspects of program development and talent acquisition. For instance, the transition from archaic tools like MS Excel to a structured platform like InternLink has been shown to improve traceability and monitoring of internships, as observed in cases like Viettel Cameroon .

2.Improved Communication and Collaboration: By providing a centralized platform, InternLink facilitates seamless communication among students, academic staff, and host organizations. This reduces miscommunication, ensures timely information dissemination, and fosters a more collaborative environment. Students can easily reach out to their supervisors, and companies can provide real-time feedback and validate weekly reports, enhancing the overall internship experience.

3.Increased Transparency and Accountability: InternLink creates a transparent record of all internship-related activities, from offer discovery and application to weekly report submission and certificate issuance. This enhances accountability for students (e.g., through activity logs and report submissions), supervisors (e.g., through report validation and feedback), and companies. The clear visibility of processes helps in identifying bottlenecks and ensuring fair practices.

4.Better Matching and Placement: Advanced platforms like InternLink incorporate sophisticated search filters and category management to match students with internship opportunities that align with their skills, academic background, and career aspirations. This leads to more meaningful placements and higher satisfaction rates for both students and companies.

5.Data-Driven Decision Making: The robust analytics dashboard and audit logs of InternLink provide institutions and companies with valuable data on program performance, user activity, and offer engagement. This data can be used to identify areas for improvement, justify resource allocation, and demonstrate the value of internship programs to stakeholders. It supports evidence-based decision-making for continuous quality enhancement.

6.Enhanced Student Experience: Ultimately, a well-designed platform like InternLink contributes to a more organized, supportive, and engaging internship experience for students. By simplifying administrative hurdles, providing easy access to resources (like the Report Guide), and facilitating structured communication and feedback, it allows students to focus more on their learning and professional development, directly impacting their satisfaction levels.

7.Compliance and Risk Management: InternLink helps institutions and companies ensure compliance with various regulations and policies related to student placements, safety, and data privacy. Centralized documentation, report validation, and audit logs reduce legal and ethical risks associated with managing a large volume of internships.

In the context of higher education in Cameroon, where challenges such as fragmented management and limited IT adoption persist , the importance of InternLink is even more pronounced. It offers a tangible solution to overcome these systemic issues, thereby elevating the quality and impact of experiential learning opportunities for students in Cameroon.

2.1.2. THE DEPENDENT VARIABLE : STUDENT SATISFACTION

Student Satisfaction in the context of internship programs refers to the positive emotional response or overall contentment experienced by students regarding various aspects of their internship. This encompasses their perceptions of the program's organization, the support received, the learning opportunities provided, the relevance of the work, and the overall administrative processes. It is a critical indicator of the quality and effectiveness of an educational experience, directly influencing student engagement, retention, and future career success.

Definition

Student satisfaction is a subjective construct, often measured by assessing the gap between a student's expectations and their actual experiences during the internship . A high level of satisfaction implies that the internship met or exceeded their expectations, leading to positive feelings and a sense of accomplishment. Conversely, dissatisfaction arises when experiences fall short of expectations, potentially leading to frustration and disengagement. In the realm of higher education, student satisfaction is increasingly recognized as a key performance indicator, reflecting the value proposition of academic programs and institutions .

Factors Affecting Student Satisfaction

Numerous factors contribute to or detract from student satisfaction with internship programs. These can be broadly categorized into several key areas:

1.Program Organization and Administration: The efficiency and clarity of the internship program's administrative processes significantly impact student satisfaction. This includes the

ease of application, clarity of guidelines, timeliness of communication, and effectiveness of placement procedures. Manual or disorganized processes often lead to frustration and dissatisfaction .

2.Supervision and Mentorship: The quality of supervision from both academic faculty and industry mentors is paramount. Effective mentors provide guidance, constructive feedback, and support, fostering a positive learning environment. A lack of adequate supervision can leave students feeling unsupported and undervalued .

3.Learning Opportunities and Skill Development: Students seek internships to gain practical experience and develop relevant skills. Satisfaction is high when the internship provides meaningful tasks, opportunities for skill application, and exposure to new knowledge. The relevance of the work to their field of study is also a crucial factor .

4.Work Environment and Culture: A positive and inclusive work environment, supportive colleagues, and a welcoming organizational culture contribute significantly to student well-being and satisfaction. Conversely, a toxic or unsupportive environment can lead to negative experiences.

5.Compensation and Benefits: While not always the primary driver, fair compensation (if applicable) and other benefits (e.g., housing, transportation support) can influence student satisfaction, particularly for those facing financial constraints.

6.Expectation Management: The alignment between a student's initial expectations of an internship and their actual experience is a powerful determinant of satisfaction, as posited by the Expectancy Disconfirmation Theory . Realistic expectations, set through clear communication and accurate program descriptions, are vital.

7.Technology Support: The availability and effectiveness of technological tools, such as InternLink, can greatly influence satisfaction. A user-friendly platform that simplifies tasks, provides easy access to information, and facilitates communication can enhance the overall experience, while a cumbersome system can be a source of frustration .

Dimensions/Measures of Student Satisfaction

Measuring student satisfaction with an internship program typically involves assessing several key dimensions. These dimensions often align with the factors affecting satisfaction and can be quantified through surveys, interviews, and feedback mechanisms. Key measures include:

1.Perceived Usefulness (PU): Derived from the Technology Acceptance Model (TAM), this dimension assesses the extent to which students believe that using InternLink (or participating in the internship) will enhance their job performance, learning, or effectiveness . For InternLink, this translates to whether students perceive the platform as helpful for finding internships, managing tasks, submitting reports, and communicating effectively.

2.Perceived Ease of Use (PEOU): Also from TAM, this measures the degree to which students believe that using InternLink is free of effort and easy to understand. A platform that is intuitive, simple to navigate, and requires minimal cognitive load will score high on PEOU, contributing to higher satisfaction .

3.Overall Program Quality: This dimension captures students' general assessment of the internship program's quality, including its structure, resources, and support mechanisms. It often reflects a holistic view of their experience.

4.Learning and Development Outcomes: This measures students' perceptions of how much they learned, the skills they acquired, and how the internship contributed to their personal and professional growth. This aligns with Kolb's Experiential Learning Theory, where reflection on concrete experiences leads to new knowledge .

5.Supervisor/Mentor Effectiveness: This assesses the quality of guidance, feedback, and support received from both academic and industry supervisors.

6.Relevance of Experience: This dimension evaluates how well the internship experience aligns with students' academic studies and career goals.

7.Administrative Support: This measures satisfaction with the administrative aspects, such as application process, documentation, and problem resolution.

By systematically measuring these dimensions, researchers and institutions can gain a comprehensive understanding of student satisfaction and identify specific areas for improvement in internship programs and their supporting management systems, particularly in the context of InternLink.

2.1.3 FULL-STACK DEVELOPMENT

Full-stack development refers to the process of developing both the frontend (client-side) and backend (server-side) components of a web application. A full-stack developer possesses expertise across all layers of software development, from user interface design to database management and server logic [3]. The frontend typically involves technologies such as HTML, CSS, and JavaScript frameworks (e.g., React, Angular, Vue.js), responsible for the visual and interactive aspects of the application that users directly interact with. The backend, on the other hand, handles server logic, database interactions, authentication, and API management, often utilizing languages like Python, Node.js, Java, or PHP, and various database systems (e.g., MySQL, PostgreSQL, MongoDB) [4]. The integration of these components ensures a cohesive and fully functional web application. Modern full-stack development often emphasizes agile methodologies, continuous integration/continuous deployment (CI/CD), and DevOps practices to ensure efficient and scalable development workflows [5].

2.1.4 EXPERIENTIAL LEARNING AND INTERNSHIPS

Internships are a critical component of experiential learning, providing students with practical, real-world experience that complements their academic studies. The National Association of Colleges and Employers (NACE) defines an internship as a form of experiential learning that integrates knowledge and theory learned in the classroom with practical application and skill development in a professional setting [6]. These experiences are vital for career readiness, allowing students to explore career paths, develop professional skills, and build networks. Effective internships are structured, supervised, and provide meaningful work that contributes to the intern's learning objectives and the employer's goals [7].

2.2 THEMATIC REVIEW

2.2.1 TRENDS IN INTERNSHIP MANAGEMENT

The landscape of internship management has evolved significantly with the advent of digital technologies. There is a growing trend towards the adoption of centralized online platforms to manage the entire internship lifecycle. These platforms aim to replace traditional, often manual, processes with automated workflows, improving efficiency for educational institutions, employers, and students alike. Modern IMS solutions often incorporate features such as personalized dashboards, real-time tracking of applications, digital submission of reports, and automated feedback mechanisms. The focus is on creating a seamless and transparent experience, reducing administrative burdens, and enhancing the quality of internship placements [1].

2.2.2 SOFTWARE DEVELOPMENT METHODOLOGIES

The development of complex web applications like InternLink often employs agile methodologies, such as Scrum or Kanban, which emphasize iterative development, flexibility, and continuous feedback. Agile approaches are particularly well-suited for projects with evolving requirements, allowing for rapid prototyping and adaptation to user needs [8]. Other methodologies, such as the Waterfall model, are more linear and sequential, often preferred for projects with well-defined and stable requirements. However, for dynamic projects like an IMS, **agile methods** offer greater responsiveness and collaboration, which are crucial for delivering a user-centric solution [9].

2.3 EMPIRICAL REVIEW

2.3.1 EXISTING INTERNSHIP MANAGEMENT SYSTEMS

Several studies have explored the development and implementation of internship management systems. Abdullah et al. (2017) [10] presented a web-based application for an internship management system, highlighting its effectiveness in managing tasks and satisfying users with its functions. Similarly, research by Kadriu and Mydyti (n.d.) [11] focused on the development of an IMS at the university level, emphasizing its role as an intermediary platform. These studies generally conclude that such systems significantly improve the efficiency of internship processes by automating tasks such as application collection, review, and management. However, many

existing systems may lack comprehensive features or may not be tailored to specific regional contexts, presenting opportunities for improvement.

2.3.2 CHALLENGES IN INTERNSHIP PLACEMENT IN DEVELOPING COUNTRIES (FOCUS: CAMEROON)

Internship placement in developing countries, including Cameroon, faces unique challenges. These often include a lack of structured internship programs, limited access to information about available opportunities, and a disconnect between academic curricula and industry demands. Research indicates that students in regions like Buea Health District, Cameroon, face challenges such as limited time and lack of support and supervision during their internships [12]. Furthermore, the informal nature of many job markets in these regions can make it difficult for students to find verified and meaningful placements. The absence of centralized platforms exacerbates these issues, leading to inefficiencies and missed opportunities. While some studies highlight the positive impact of internship programs on labor market indicators in Sub-Saharan Africa, including Cameroon, the underlying challenges in access and management persist [13]. InternLink aims to directly address these regional specificities by providing a tailored and centralized solution.

2.4 MOBILE TECHNOLOGIES

Based on the review of existing systems and related studies, it is clear that many platforms aim to manage academic and professional internships. However, they differ in how they are built, the technologies they use, the environments in which they operate, and their overall system design.

To address the limitations identified in these platforms, **InternLink** is developed as a responsive web application that simplifies and centralizes internship management. The platform enables educational institutions to track student internship progress, allows organizations to publish internship opportunities, and helps students easily search for and apply to available placements. Its main goal is to reduce administrative workload, improve access to internship opportunities, and make the internship process more efficient, organized, and transparent.

To support these objectives, the system is built using modern and reliable web and mobile technologies, as presented below:

a) Integrated Development Environment: Visual Studio Code

Visual Studio Code (VS Code) is a free, open-source-driven, lightweight but exceptionally powerful source-code editor developed by Microsoft. Built on the Electron framework, it combines the speed of a streamlined text editor with the advanced features of a traditional Integrated Development Environment (IDE), such as debugging, syntax highlighting, and intelligent code completion.

Advantages & Limitations of VS Code

Category	Advantage	Limitations
1. Resource & Scope	Extensive Extension Ecosystem: Features a massive marketplace allowing developers to add tailored languages, linters, and tools.	Not a Full IDE Out-of-the-Box: Requires manual configuration and installation of extensions to match the native power of full IDEs like WebStorm.
2. Performance	Lightweight Footprint: Launches incredibly fast and consumes far fewer initial system resources than heavy IDEs.	High Memory Consumption Under Load: Being built on Electron, running dozens of active extensions can quickly drain system RAM.
3. Git Integration	Native Version Control: Built-in Git GUI allows staging, committing, pushing, and resolving conflicts without using the terminal.	Basic Native Git Features: Advanced Git branching workflows and deep history visualization still require third-party tools.

Category	Advantage	Limitations
4. Development Tools	Advanced IntelliSense: Offers smart completions out-of-the-box based on variable types, function definitions, and imported modules.	Extension Conflicts: Installing multiple plugins for the same language can cause performance stuttering or conflicting autocomplete bugs.
5. Customization	Granular Configuration: Allows deep personalization of themes, keybindings, and workspaces via an accessible UI or JSON files.	Complex Settings Management: Navigating thousands of lines of configuration settings in JSON can be overwhelming for beginners.

Why Visual studio code and no other IDEs?

1. **Native TypeScript Synergy:** Developed by Microsoft, VS Code offers unmatched, built-in compiler diagnostics and auto-imports for TypeScript without requiring external plugins.
2. **Cost-Effectiveness:** It is completely free and open-source, eliminating the expensive licensing fees associated with alternatives like JetBrains WebStorm.
3. **Tailwind CSS Integration:** The official Tailwind IntelliSense extension provides real-time class autocompletion and linting, which is vital for building the Internlink frontend quickly.
4. **Integrated Terminal Ecosystem:** Eliminates window-switching by allowing developers to run Next.js development servers, Prisma migrations, and Git operations directly inside the workspace.
5. **Universal Ecosystem Dominance:** Its massive community ensures that any bug, configuration issue, or tooling requirement has an immediately available solution online.
6. **Cross-Platform Settings Sync:** Keeps settings, extensions, and snippets synchronized perfectly across different development machines via a simple GitHub or Microsoft login.

7. **Prisma Schema Support:** The dedicated Prisma extension provides syntax highlighting, formatting, and validation for schema.prisma files out of the box.
8. **Low System Overhead Relative to Power:** It provides a middle ground—vastly faster and lighter than full IDEs, yet significantly more functional than minimalist text editors like Sublime Text.
9. **Built-in Workspace Workspaces:** Allows developers to split screens, manage multiple root folders simultaneously, and save environment profiles tailored specifically to the Internlink project structure.
10. **Industry Standard Alignment:** Operating within the most widely used editor globally ensures that the project structure remains highly collaborative and easily readable by other developers.

2.4.1 TECHNICAL STACK

1. Next.js 16 (Frontend Framework)

Next.js 16 is a production-ready React meta-framework optimized for server-side operations and static site generation. Utilizing the modern App Router architecture, it handles routing, data fetching, rendering, and optimization natively, allowing developers to seamlessly blend client-side interactivity with server-side performance.

Advantages

- **Hybrid Rendering Options:** Offers Server-Side Rendering (SSR) and Static Site Generation (SSG) out of the box, ensuring lightning-fast initial page loads for internship listings.
- **Unified Full-Stack Architecture:** Incorporates API Routes, allowing developers to write backend server code and endpoints directly alongside frontend UI components.
- **Automatic Code Splitting:** Breaks down JavaScript bundles into smaller pieces per route, loading only the necessary code to optimize client-side speed.
- **Built-in Performance Optimizations:** Native components automatically handle image scaling, font loading, and script prioritization to maximize Core Web Vitals.

- **React Server Components (RSC):** Fetches data directly on the server, significantly reducing the volume of JavaScript sent to the user's browser

Disadvantages

- **Steep Architectural Learning Curve:** Managing data flow between Server Components and Client Components introduces architectural complexity.
- **Vendor Optimization Bias:** While open-source, Next.js is heavily optimized for Vercel's cloud hosting, making self-hosting on traditional servers more technically demanding.
- **Frequent Breaking API Changes:** Rapid evolutionary updates between major versions (like the shift to the App Router) can break legacy codebase paradigms.
- **High Development Build Overhead:** The sophisticated compilation, linting, and bundling pipelines can cause slow hot-reloading times on lower-spec hardware.
- **State Management Friction:** Passing state from server-rendered layouts down to client-side interactive forms requires highly disciplined architecture.

Reasons for Choosing Next.js 16 over Vite or Create React App

1. **Dynamic SEO for Job Postings:** Next.js uses server-side rendering to expose internship openings directly to search engine crawlers, a feature client-side React apps fail to do efficiently.
2. **Zero-Configuration Routing:** The file-system-based App Router replaces complex, manual React Router setups with highly predictable directory structures.
3. **Elimination of a Separate Backend Server:** Next.js API routes host the application's backend logic directly, eliminating the need to spin up and maintain a separate Express.js server.
4. **Native Route Protection:** Server-side middleware easily intercepts incoming requests to instantly verify student or coordinator authentication tokens before rendering views.

5. **Optimized Resource Delivery:** Built-in components handle image caching and font optimization natively, ensuring Internlink remains accessible on slower mobile networks.

2. TypeScript (Programming Language)

TypeScript is an open-source, strongly typed superset of JavaScript developed by Microsoft. It injects a compile-time static type system into JavaScript, allowing developers to define explicit data structures and interfaces that catch software defects before code execution.

Advantages

- **Compile-Time Error Elimination:** Flags semantic errors, type mismatches, and structural flaws during development rather than risking crashes in production.
- **Self-Documenting Codebases:** Explicit interfaces define exactly what data an object contains, making the codebase highly legible without sprawling documentation files.
- **Unmatched Autocompletion:** Empowers IDEs to deliver context-aware code completions, property suggestions, and parameter definitions in real time.
- **Safe and Predictable Refactoring:** Allows safe structural changes across hundreds of files simultaneously; the compiler instantly identifies every affected line of code.
- **Strict Control Flows:** Enforces predictable variables via strict null checking and type guards, removing common runtime errors like "undefined is not a function."

Disadvantages

- **Initial Development Velocity Penalty:** Writing explicit types, interfaces, and generic bounds increases boilerplate and slows down initial prototyping.
- **Compilation Time Overhead:** Requires an extra compilation step to transpile TypeScript down into browser-readable JavaScript, which can extend build durations.
- **Steep Learning Curve for Teams:** Mastering advanced concepts like generics, utility types, and strict union configurations requires significant technical training.
- **Friction with Legacy Ecosystems:** Integrating older, untyped JavaScript libraries requires manually creating custom definition files (.d.ts).

- **False Sense of Runtime Security:** Type validation occurs strictly during compilation; it cannot natively prevent unexpected schema malformations from external API inputs without runtime validation helpers.

Reasons for Choosing TypeScript over Plain JavaScript

1. **Bulletproof Application Workflows:** Guarantees that an internship submission payload strictly contains all required fields (`studentId`, `companyId`,) before it ever hits the database layer.
2. **Seamless Prisma Synergy:** Prisma maps database models directly into native TypeScript types, creating an unbroken chain of type-safety from the database layout to the frontend form.
3. **Accelerated Team Onboarding:** Classmates and contributors can open any component and immediately understand expected data shapes via IDE tooltips.
4. **Resilience to Code Rot:** As the Internlink platform expands with new features, TypeScript ensures older modules remain compatible with new architectural updates.
5. **Modern JavaScript Standardization:** Enables access to cutting-edge ECMAScript features safely, handling polyfills and compatibility targets automatically during compilation.

3. Tailwind CSS 4 (Styling Framework)

Tailwind CSS 4 is a utility-first CSS framework designed to construct custom user interfaces directly inside markup elements. Built around a lightning-fast, native Rust compiler engine, version 4 uses modern CSS features like cascade layers and variables to deliver highly responsive, unified design systems without writing conventional stylesheets.

Advantages

- **Rapid UI Assembly Engine:** Accelerates development speed by eliminating the need to invent class names or jump between JSX files and separate CSS stylesheets.

- **Ultra-Lean Production CSS Files:** The build engine scans the project and purges all unused styles, outputting a highly optimized, minified CSS file that is rarely larger than 10KB.
- **Design Token Consistency:** Enforces strict visual uniformity across spacing, color scales, typography, and responsive breakpoints out of the box.
- **Zero Runtime Overhead:** Compiles down to standard, raw utility CSS classes during the build step, bypassing the browser processing penalties of CSS-in-JS libraries.
- **Seamless Responsive Utilities:** Implements responsive design directly inside components using intuitive prefixes (e.g., `md:flex-row`, `lg:grid-cols-3`).

Disadvantages

- **Cluttered Component Markup:** Embedding dozens of utility classes directly inside class attributes can make JSX code look dense and difficult to read.
- **Absence of Pre-built Elements:** Does not provide styled components out of the box; UI elements like modals, accordions, and buttons must be built entirely from scratch.
- **High Initial Memorization Phase:** Developers must invest time into learning Tailwind's specific utility naming conventions and syntax rules.
- **Complex Conditional Style Application:** Dynamically toggling styles based on complex React component states requires utility wrappers like `clsx` or `tailwind-merge`.
- **Disciplinary Dependence:** Without team style standards, layouts can quickly become visually fragmented if developers arbitrarily mix custom values with default spacing classes.

Reasons for Choosing Tailwind CSS 4 over Bootstrap or Styled Components

1. **Unmatched Performance via the Rust Compiler:** The updated v4 Rust engine compiles style modifications nearly instantaneously, greatly improving the local development experience.
2. **Absolute Freedom from Generic Visual Identifiers:** Unlike Bootstrap, which forces identifiable button shapes and navbar structures, Tailwind allows Internlink to maintain a completely tailored aesthetic.

- 
3. **Zero JavaScript Execution Penalties:** Unlike Styled Components or Emotion, Tailwind doesn't require runtime JavaScript evaluation to parse styles, which saves battery life and improves mobile loading speeds.
 4. **Frictionless Dark Mode Integration:** Implementing a platform-wide system theme toggle is as simple as prepending a utility class (e.g., `dark:bg-slate-900`).
 5. **Intuitive Responsive Adaptability:** Building a responsive layout that scales down from a coordinator's desktop dashboard to a student's mobile viewport requires only simple class inline prefixes.

4. Zustand (State Management)

Zustand is a minimalistic, fast, and scalable state management library for React applications. Operating via a simplified, hook-based architecture, it provides a centralized store that manages global application states cleanly while bypassing the heavy boilerplate, context wrappers, and rendering inefficiencies of older alternatives.

- **Minimalist Developer Footprint:** Requires zero complex providers, actions, reducers, or massive boilerplate configurations to instantiate a functioning global store.
- **High Performance Rendering Optimization:** Components select precisely which parts of the global state they care about, completely preventing unnecessary re-renders across the rest of the application.
- **Clean Hook-Based API:** State consumption uses standard React hook patterns, making state integration intuitive and native to modern React development.
- **Decoupled Architecture:** Stores are written in pure JavaScript/TypeScript, allowing state mutations to be triggered easily outside of UI components.
- **Native Ephemeral Persistence Middleware:** Includes built-in utilities to automatically persist selected state data directly to `localStorage` or `sessionStorage` with a single wrapper.

Disadvantages

- **Lack of Opinionated Design:** The absence of structural rules can lead to disorganized, scattered codebases if developers don't manually enforce strict state mutation guidelines.
- **Smaller Ecosystem Tooling:** Lacks the extensive developer tool ecosystem, middleware options, and historical tracking extensions enjoyed by Redux.
- **Manual Action Architecture Requirement:** Updating state safely requires developers to write explicit setter functions within the store to maintain predictability.
- **Server-Side Rendering Context Risks:** Because stores exist outside the React component tree, managing isolated user sessions on a Next.js server requires specific configuration to prevent data leaks between requests.
- **Traceability Challenges:** Lacks built-in, granular state mutation history timelines, making tracking complex multi-step state changes harder to debug without custom logging.

Reasons for Choosing Zustand over Redux Toolkit or React Context API

1. **Elimination of Redux Boilerplate:** Zustand eliminates the need to configure slices, action types, thunks, and providers, allowing developers to set up a functional store in just a few lines of code.
2. **Bypassing Context Re-render Bottlenecks:** React Context triggers updates across all consumer components whenever any part of its value object changes; Zustand isolates updates strictly to components tracking that specific field.
3. **Simplifying Complex Multi-Step Forms:** Perfect for caching data across the multi-page student application wizard without constantly executing database writes.
4. **Ultra-Lightweight Library Impact:** Adds less than 2KB to the final application shipping bundle, keeping the overall footprint minimal.
5. **Native TypeScript Support:** Built natively with TypeScript typing inference, removing the need to manage separate typed selectors or dispatch overrides.

5. SQLite (via Prisma ORM) (Database & Data Access)

SQLite is a lightweight, serverless, self-contained relational database engine that saves entire datasets as a single file directly on disk. It is managed here using **Prisma ORM**, a modern, type-

safe Object-Relational Mapper that translates declarative schema configurations into type-safe database queries.

Advantages

- **Zero Infrastructure Configuration:** SQLite stores everything inside a single local file, eliminating the need to set up, configure, and maintain an external database server like PostgreSQL during development.
- **End-to-End Type Safety:** Prisma automatically reads the database structure and generates matching TypeScript clients, guaranteeing full type validation across all queries.
- **Automated Migration Handling:** Prisma Migrations seamlessly generate and execute standard SQL scripts automatically based on updates made to the `schema.prisma` file.
- **Extremely Fast Read Latency:** Because the database file resides locally on the application server, read operations skip network latency entirely.
- **Prisma Studio Visual Dashboard:** Includes a clean, built-in browser UI tool that lets developers view, filter, and edit database records visually without writing manual SQL scripts.

Disadvantages

- **Concurrency Write Bottlenecks:** SQLite uses file-level locking for transactions, which means heavy, simultaneous database writes will queue up and block other operations.
- **Lack of Advanced Relational Features:** Missing enterprise database capabilities like native full-text search engines, custom user roles, and advanced geospatial query support.
- **No Horizontal Scaling:** Being a single local disk file, SQLite cannot scale out across multiple server instances or distributed cloud clusters.
- **Strict Ephemeral Server Storage Limits:** If hosted on server environments with ephemeral file systems (like basic serverless functions), the database file will be wiped clean on every cold restart unless paired with persistent disk volumes.

- **Abstraction Performance Pitfalls:** Prisma's automated queries can occasionally generate complex or inefficient underlying SQL joins that mask performance issues unless explicitly profiled.

Reasons for Choosing SQLite + Prisma over PostgreSQL + TypeORM or MongoDB

1. **Simplified Infrastructure for Local Execution:** Using SQLite allows classmates, examiners, and developers to clone the Internlink project repository and launch it immediately without installing local database engines.
2. **Declarative Database Schemas:** Prisma models database entities cleanly using an intuitive, human-readable configuration language, completely bypassing the complex decorator setups required by TypeORM.
3. **Seamless Future Migration Path:** If Internlink needs to scale up to an enterprise production environment, migrating from SQLite to PostgreSQL requires changing only a single provider word in the `schema.prisma` file.
4. **Guaranteed Local Data Isolation:** Keeps development testing workflows cleanly self-contained on the programmer's local machine, preventing unexpected cloud database hosting bills or configuration errors.
5. **Elimination of "Object-Relational Impedance Mismatch":** Prisma returns query results as clean, plain JavaScript objects that map perfectly to frontend UI properties without needing extra parsing steps.

2.5 SDLC MODEL

The **Software Development Life Cycle (SDLC)** is a highly structured, conceptual framework that outlines the distinct phases required to design, develop, test, deploy, and maintain a software application. It functions as an engineering roadmap, establishing systematic procedures that guarantee software quality, structural integrity, and alignment with organizational requirements.

By breaking down the complex engineering lifecycle into manageable phases, an SDLC ensures that project teams operate under predictable timelines, clear resource allocations, and rigorous quality control baselines from initial conception through to final system decommissioning.

2.5.1 IMPORTANCE OF SDLC

Implementing a clearly defined SDLC is vital for the success of any enterprise software system. Its importance stems from several core operational benefits:

- **Risk Mitigation:** By enforcing structured phases, an SDLC exposes technical bottlenecks, architectural flaws, and scope mismatches early in the project timeline, preventing late-stage systemic failures.
- **Predictable Resource Management:** It allows project managers to allocate developer bandwidth, budget assets, and hardware infrastructure precisely, avoiding resource exhaustion or project delays.
- **Enhanced Quality Assurance:** Testing is integrated as a fundamental phase rather than a rushed afterthought, guaranteeing that code correctness, security parameters, and performance targets are met.
- **Transparent Progress Tracking:** Establishes concrete milestones and deliverables for each phase, allowing stakeholders, academic supervisors, and development teams to accurately gauge true project velocity.
- **Standardized Documentation:** Ensures that codebases are backed by comprehensive architectural designs, system schemas, and user manuals, facilitating future maintenance and seamless onboarding of new developers.

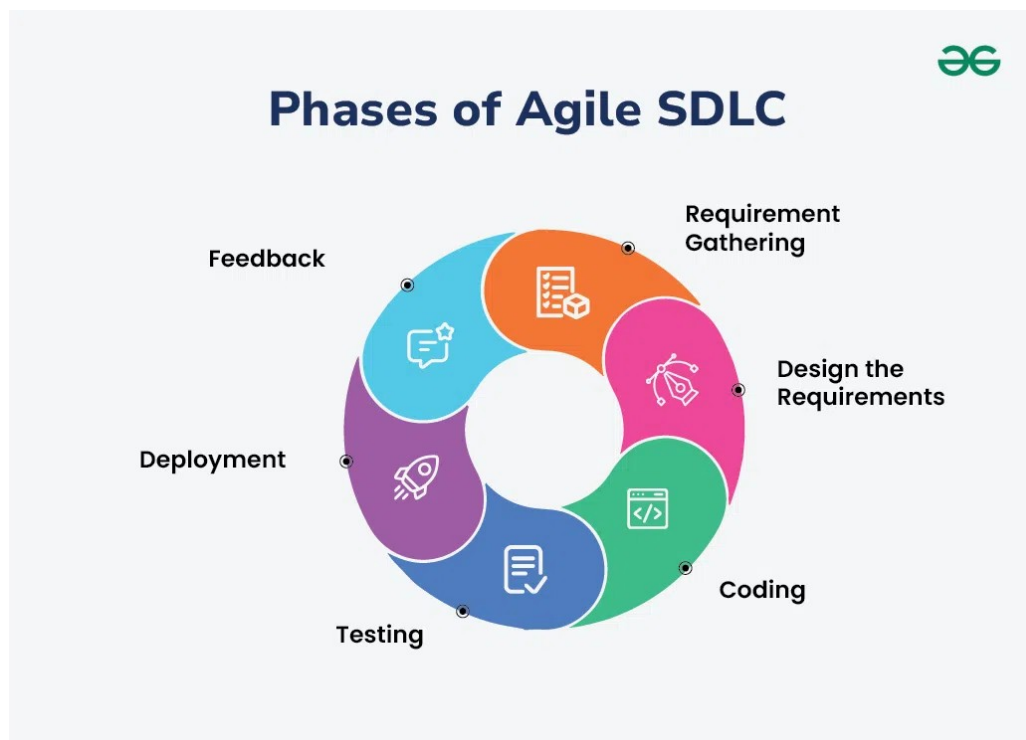
2.5.2 Agile Model

The **Agile Model** is an iterative and incremental software development philosophy centered around adaptability, rapid feedback loops, and cross-functional collaboration. Unlike rigid, linear frameworks, Agile rejects the premise that software requirements can be perfectly fixed up front. Instead, it breaks the application down into minor, functional components developed over short time frames known as **sprints** (typically lasting 1 to 4 weeks). Each sprint produces a fully

functional, testable software increment, allowing the system to evolve organically based on real-world testing and stakeholder critique.

As demonstrated in the figure below, the Agile lifecycle moves away from a monolithic start-to-finish line and instead cycles continuously through specialized phases to constantly refine the application.

Figure 1: Agile model



Source : [geeksforgeeks](https://www.geeksforgeeks.org/agile-model/)

1. Requirement Gathering

This initial phase focuses on understanding the core problem and defining what needs to be built. It involves discussing the needs of the end-users to figure out exactly what features the software requires. For example, identifying that a business is struggling with manual data entry bottlenecks and determining they need specific features to track stock status, IMEI numbers, and daily sales statistics.

2. Design the Requirements

Once the requirements are clear, they are translated into a technical blueprint. This involves planning the application's architecture, mapping out the database schema (such as planning the necessary SQL tables for inventory), and creating wireframes for the user interface. It sets the roadmap before any actual programming begins.

3. Coding

This is the active development phase where the software is built. Developers use the design blueprints to write the actual logic and build the application's features. This involves opening up an editor like VS Code and writing the front-end and back-end logic, often keeping related code co-located and utilizing a straightforward, reliable stack like HTML, CSS, JavaScript, and PHP to bring the project to life.

4. Testing

Before the software reaches the user, it must be rigorously checked for quality. This phase involves hunting down bugs, verifying that the database stores information correctly, and ensuring all the features built during the coding phase work exactly as intended without crashing.

5. Deployment

Once the code is tested and stable, the application is released into a live environment. The project is moved from a local development setup to a web server where the actual end-users can access and start using the application.

6. Feedback

After the software is deployed, users interact with it and provide their thoughts. They might spot a minor bug, find a workflow confusing, or request a new feature (like integrating local AI automation for data processing). This feedback is collected and flows directly back into the **Requirement Gathering** phase, starting the cycle all over again to continuously improve the software.

Advantages & Disadvantages of the Agile Model

No.	Advantage (Stated & Explained)	Disadvantage (Stated & Explained)
1	High Adaptability to Shifting Requirements: Requirements can change mid-project without derailing the entire timeline; changes are cleanly prioritized in the next sprint backlog.	High Exposure to Scope Creep: Without rigid parameters, continuous feedback can cause the project scope to balloon, pushing out ultimate deadlines.
2	Rapid and Early Value Delivery: By deploying operational increments at the end of every sprint, users interact with a working application much sooner.	Unpredictable Final Timelines and Budgets: Because the final product is evolving dynamically, defining exact completion dates or fixed total costs up front is difficult.
3	Continuous Feedback Loops: Frequent sprint reviews ensure that user feedback is immediately integrated, aligning the application closely with user needs.	Heavy Organizational Commitment: Demands continuous, highly active collaboration from developers, stakeholders, and users, which can drain team energy.
4	Substantial Quality Enhancement: Integrating continuous testing and development cycles means code defects are found and corrected immediately when they are cheap to fix.	Risk of Minimal Documentation Architecture: The core focus on working software over paperwork can result in an undocumented codebase that is difficult for new developers to parse.
5	Transparent Technical Progress: Stakeholders can physically view functional software progress during sprint reviews, removing guesswork around true project status.	Dependence on Highly Disciplined Teams: Requires experienced, self-motivated developers who can manage their workloads efficiently without rigid top-down management.

2.6 PRESENTATION OF INTERNSHIP ACTIVITIES

During the internship period at Lumia Group International LLC and our field study at ISTAG OMNISPORT , a comprehensive analysis of the Internship management system was conducted. This involved direct observation, interviews with key personnel and a review of existing documentation. The primary objective was to identify pain points, inefficiencies, and opportunities for improvement through a web application solution.

Week	Key Objectives	Activities
Week 1: Onboarding & Project Kick-off	- Understand company culture and tools. - Familiarize with project scope and client requirements. - Set up development environment.	- Engaged in comprehensive orientation sessions and was introduced to key team members, fostering an understanding of the startup's operational dynamics. - Conducted a thorough review of existing project documentation, including the client brief and initial scope definitions, to grasp the project's foundational aspects. - Successfully installed and configured essential development tools, including Integrated Development Environments (IDEs), Software Development Kits (SDKs), and version control systems (e.g., Git), ensuring a ready development environment. - Participated in initial discussions concerning the proposed mobile application architecture and the selection of appropriate technology stacks.
Week 2: Requirement s Analysis & UI/UX Design	- Deep dive into functional and non-functional requirements. - Contribute to UI/UX wireframing and prototyping.	- Actively participated in client meetings to gather detailed functional and non-functional requirements, ensuring a clear understanding of user needs and system constraints. - Analyzed user stories and use cases specifically tailored for various hospital staff roles, such as

		doctors, nurses, and administrative personnel, to define application functionalities. - Collaborated closely with UI/UX designers on the creation of wireframes and mockups for critical application screens, including patient registration, appointment scheduling, and medical records management. - Conducted research into best practices for user interface and user experience design within the healthcare mobile application domain, aiming for intuitive and efficient user interaction.
Week 3: Database Design & API Integration Planning	- Design database schema for the application. - Plan API endpoints and data flow.	- Developed a comprehensive relational database schema to effectively manage various data entities, such as patient information, appointment schedules, and staff details. - Documented detailed API specifications, outlining endpoints, request/response formats, and authentication mechanisms for seamless backend communication. - Researched and incorporated secure data handling practices and compliance standards (e.g., HIPAA, GDPR) to ensure the protection of sensitive patient information. - Initiated the setup of a local database environment for development and testing purposes.
Week 4: Core Feature Development Student Management	- Implement core student registration and profile management features.	- Developed and implemented user authentication and authorization modules, ensuring secure access to the application. - Created robust patient registration forms with comprehensive data validation to maintain data

		integrity. - Implemented functionalities for displaying and editing patient profiles, allowing for efficient management of patient information. - Integrated these features with mock API endpoints to simulate data persistence and interaction with the backend.
Week 5: Core Feature Enterprise Management	- Implement core Enterprise registration and profile management features.	- Developed and implemented user authentication and authorization modules, ensuring secure access to the application. - Created robust patient registration forms with comprehensive data validation to maintain data integrity. - Implemented functionalities for displaying and editing patient profiles, allowing for efficient management of patient information. - Integrated these features with mock API endpoints to simulate data persistence and interaction with the backend.
Week 6: Module Development of attestations	- Implement features for viewing and managing rendering attestations	- Developed secure access protocols for Interns evaluation and status, ensuring confidentiality and compliance. - Implemented a comprehensive weekly report rendering - Prioritized data privacy and feedbacks in all aspects of handling sensitive internship informations.
Week 7: Testing & Bug Fixing - Alpha	- Conduct unit and integration testing. - Identify and fix bugs.	- Wrote and executed extensive unit tests for all developed modules to ensure individual component functionality.

Release		- Performed thorough integration tests to verify seamless data flow and interaction between different application components. - Utilized a project management tool for logging and tracking identified bugs, facilitating efficient resolution. - Collaborated actively with the Quality Assurance (QA) team during the initial alpha testing phase to gather feedback and identify further issues.
Week 8: UI/UX Refinement & Performance Optimization	- Refine user interface based on feedback. - Optimize application performance.	- Implemented significant UI/UX refinements based on feedback received from internal reviews and the alpha testing phase, enhancing user satisfaction. - Conducted profiling of application performance to identify and address potential bottlenecks, ensuring smooth operation. - Optimized code and resource utilization to improve application responsiveness and overall efficiency. - Ensured cross-device compatibility and responsiveness across various mobile platforms and screen sizes.
Week 9: Security Enhancements & Deployment Preparation	- Implement security best practices. - Prepare for application deployment.	- Reviewed and significantly enhanced application security features, including data encryption and secure API call implementations, to protect sensitive information. - Prepared the application build for deployment across different platforms (iOS/Android), adhering to platform-specific guidelines. - Assisted in the comprehensive documentation required for submission to various app stores.

		- Conducted final rounds of testing and quality assurance checks to ensure the application's readiness for launch.
Week 10: Final Presentation & Testimonial	- Present project contributions. - Receive internship testimonial.	- Prepared and delivered a comprehensive final presentation, effectively showcasing individual contributions to the project and highlighting acquired skills. - Engaged in feedback sessions with mentors and team leads to discuss performance and areas for growth. - Received an official internship completion certificate and a testimonial, acknowledging successful participation and contributions. - Facilitated knowledge transfer and documented completed work to ensure continuity for future team members.

CHAPTER THREE: METHODOLOGY

3.1 STUDY DESIGN

The study adopted a mixed-methods research design, specifically an explanatory sequential design, where quantitative data was collected and analyzed first, followed by qualitative data collection and analysis to help explain or elaborate on the quantitative results . This approach was chosen to provide a comprehensive understanding of the impact of the IMS on student satisfaction. The quantitative phase involved assessing student satisfaction with InternLink using structured questionnaires, while the qualitative phase involved interviews to gain deeper insights into their experiences and perceptions of the platform's specific features.

The primary research method employed was a survey research method for the quantitative phase, which allowed for the collection of data from a large sample of students regarding their perceptions of InternLink. This method is suitable for describing the characteristics of a large population and examining relationships among variables . For the qualitative phase, a case study approach was utilized, focusing on the implementation of InternLink within a specific higher education institution in Yaounde. This allowed for an in-depth exploration of the context and processes involved in InternLink's adoption and its influence on student satisfaction.

The research approach was largely deductive, moving from general theoretical frameworks (such as the Technology Acceptance Model and Expectancy Disconfirmation Theory) to specific observations and empirical testing of hypotheses. However, the qualitative component also incorporated elements of an inductive approach, allowing for the emergence of new themes and insights from the participants' experiences, thereby enriching the interpretation of the quantitative findings . This dual approach ensured that the study was both theoretically grounded and empirically rich, providing a nuanced understanding of the research problem.

The choice of a mixed-methods research design was particularly justified for this study due to the multifaceted nature of investigating a comprehensive platform like InternLink and its impact on student satisfaction. While quantitative methods are excellent for measuring the extent of satisfaction and identifying statistical relationships between variables (e.g., perceived usefulness and satisfaction), they often fall short in providing the 'why' and 'how' behind these relationships. Qualitative methods, on the other hand, excel at exploring complex phenomena in depth,

capturing nuanced perspectives, and uncovering underlying reasons for observed patterns . By combining both, the study aimed to achieve both breadth and depth of understanding.

The explanatory sequential design specifically allowed for the quantitative phase to establish generalizable patterns and test hypotheses derived from established theories like TAM and EDT. For instance, the survey data could reveal whether a statistically significant relationship exists between the perceived ease of use of InternLink and student satisfaction. Following this, the qualitative phase, through interviews, could then explore why students found certain aspects of InternLink (like the report submission or application tracking) easy or difficult to use, or how these perceptions influenced their overall satisfaction. This sequential approach ensures that the qualitative findings are directly informed by the quantitative results, providing richer interpretations and a more comprehensive narrative .

The survey research method for the quantitative phase was selected for its efficiency in collecting data from a relatively large number of students across different higher education institutions in Yaounde. This method is cost-effective and allows for the standardization of data collection, minimizing researcher bias and enhancing the reliability of the findings. The use of structured questionnaires with Likert-scale items enabled the quantification of subjective perceptions, making them amenable to statistical analysis .

For the qualitative phase, the case study approach was deemed appropriate because it allowed for an intensive, holistic description and analysis of a single unit (the implementation of an IMS within a specific institution) or a small number of units. This method is particularly valuable when the research aims to understand a contemporary phenomenon within its real-world context, especially when the boundaries between phenomenon and context are not clearly evident . By focusing on a specific institution in Yaounde, the study could explore the unique contextual factors, challenges, and successes associated with InternLink adoption in that particular environment, providing rich, context-specific insights that would be difficult to obtain through broader surveys alone.

The integration of deductive and inductive approaches further strengthened the study design. The deductive application of TAM and EDT provided a theoretical lens to formulate hypotheses and structure the quantitative data collection. This ensured that the study was grounded in established academic knowledge. Simultaneously, the inductive nature of the qualitative analysis allowed for emergent themes and unexpected findings to surface from the interview data. This flexibility is

crucial in exploratory research, especially in contexts where prior empirical work is limited, such as the specific application of platforms like InternLink in Cameroonian higher education. This iterative process of moving between theory and data allowed for a more robust and comprehensive understanding of the research problem.

3.2. STUDY POPULATIONS AND SAMPLING TECHNIQUES

3.2.1. TARGET POPULATION

The target population for this study comprised all undergraduate students enrolled in higher education institutions in Yaounde, Cameroon, who had completed or were currently undertaking an internship during the 10-week study period. This population was chosen because they were the direct users of the internship programs and would potentially interact with InternLink, making their experiences and satisfaction central to the research objectives. Additionally, the population included academic staff (internship coordinators and faculty supervisors) and representatives from host organizations involved in managing these internships, as their perspectives were crucial for understanding the challenges and requirements for an effective platform like InternLink.

Students, as the primary end-users of InternLink and the direct beneficiaries (or victims) of internship programs, were central to assessing satisfaction. Their experiences, perceptions of usefulness, and ease of use of InternLink were critical for the quantitative phase. Academic staff, on the other hand, provided insights into the administrative and pedagogical aspects of internship management. Their perspectives were crucial for understanding the 'system' side of InternLink and the broader context of internship management. Similarly, host organization representatives, who provide the actual internship placements and often interact with InternLink (e.g., posting opportunities, evaluating interns), offered an external perspective on the platform's effectiveness and the overall internship process. Their feedback was vital for ensuring InternLink met the needs of all parties involved in the internship ecosystem. By encompassing these diverse groups, the study aimed to gather a comprehensive set of perspectives, ensuring that the developed platform was not only student-centric but also institutionally viable and employer-friendly.

3.2.2. SAMPLE SIZE AND SAMPLING TECHNIQUE

For the quantitative phase, a convenience sampling technique was primarily used to select students for the survey. This non-probability sampling method involved selecting participants who were readily available and willing to participate in the study. Given the logistical constraints

and the specific focus on students engaged in internships within the defined geographical and time scope, convenience sampling allowed for practical data collection. A total of [.....] students were targeted for the survey. This sample size was deemed sufficient to capture a representative range of perceptions regarding InternLink and student satisfaction, allowing for statistical analysis of the quantitative data.

For the qualitative phase, a purposive sampling technique was employed. This involved intentionally selecting a smaller number of participants (e.g., 10-15 students, 3-5 academic staff, and 2-3 host organization representatives) who possessed specific knowledge and experiences relevant to the research questions. The criteria for selection included students who had actively used InternLink, academic staff directly involved in internship coordination, and host organization representatives who regularly engaged with interns. This technique ensured that rich, in-depth insights were gathered to elaborate on the quantitative findings and explore emergent themes related to InternLink and student satisfaction.

The small sample size (e.g., 10-15 students, 3-5 academic staff, and 2-3 host organization representatives) was justified by the goal of achieving data saturation, where no new themes or insights emerge from additional interviews. This ensured that the qualitative data provided rich, detailed narratives that could effectively explain and contextualize the quantitative findings, offering a deeper understanding of the complexities involved in InternLink adoption and student satisfaction.

3.3. DATA COLLECTION PROCEDURE

The data collection procedure was executed in two distinct phases, aligning with the mixed-methods design. The initial phase focused on quantitative data collection, followed by a qualitative phase.

Quantitative Data Collection: Structured questionnaires were the primary instrument for collecting quantitative data from students. These questionnaires were designed based on established scales for measuring perceived usefulness, perceived ease of use (from TAM), and overall student satisfaction with internship programs. The questionnaire included sections on demographic information, expectations regarding internships, experiences with InternLink and satisfaction levels across various dimensions (e.g., administrative support, learning outcomes,

supervision). The questionnaires were administered electronically via Google Forms to students who met the sampling criteria. Participants were provided with clear instructions and assured of anonymity and confidentiality. Data collection for this phase was conducted over a period of two weeks.

Qualitative Data Collection: Following the analysis of the quantitative data, semi-structured interviews were conducted with selected students, academic staff, and host organization representatives. An interview guide was developed to explore themes that emerged from the quantitative findings, as well as to delve deeper into the challenges of internship management, the specific functionalities desired in InternLink, and detailed experiences with the platform. The interviews were conducted face-to-face or via secure online video conferencing platforms, depending on the availability and preference of the participants. All interviews were audio-recorded with the explicit consent of the participants and subsequently transcribed verbatim for analysis. This phase of data collection spanned one week.

3.4. DATA ANALYSIS PROCEDURE AND ESTIMATION TECHNIQUES

The data collected from both quantitative and qualitative phases underwent rigorous analysis to address the research questions and test the hypotheses.

Quantitative Data Analysis: The quantitative data obtained from the student surveys were analyzed using Statistical Package for the Social Sciences (SPSS) software. Descriptive statistics, including frequencies, percentages, means, and standard deviations, were used to summarize the demographic characteristics of the sample and to describe the students' perceptions of InternLink and their satisfaction levels. These descriptive insights formed the basis for understanding the general perceptions of InternLink and student satisfaction within the sample. To test the research hypotheses, correlation analysis was employed to examine the strength and direction of the relationships between perceived usefulness, perceived ease of use, and student satisfaction. Specifically, Pearson product-moment correlation coefficients were calculated. Furthermore, simple linear regression analysis was utilized to determine the extent to which perceived usefulness and perceived ease of use predicted student satisfaction. This allowed for the estimation of the impact of the independent variables on the dependent variable, providing statistical evidence for the relationships hypothesized in the study.

Qualitative Data Analysis: The transcribed interview data were analyzed using thematic analysis. This involved a systematic process of reading and re-reading the transcripts to identify recurring patterns, themes, and categories related to the research questions. The steps included familiarization with the data, generation of initial codes, searching for themes, reviewing themes, defining and naming themes, and producing the report . This inductive approach allowed for a rich, in-depth understanding of the participants' experiences and perspectives, providing contextual explanations for the quantitative findings and uncovering nuances that might not have been captured by the survey alone.

3.5. ETHICAL CONSIDERATION

Throughout the research process, strict ethical guidelines were adhered to ensure the protection and well-being of all participants and the integrity of the study. Prior to commencing data collection, ethical approval was obtained from the relevant institutional review board of the participating higher education institution. Key ethical considerations included:

- Informed Consent:** All participants were provided with a clear and comprehensive explanation of the study's purpose, procedures, potential risks, and benefits. They were informed that their participation was entirely voluntary and that they had the right to withdraw at any point without penalty. Written informed consent was obtained from all participants before their involvement in the survey or interviews.
- Anonymity and Confidentiality:** Measures were taken to ensure the anonymity of survey respondents and the confidentiality of interview participants. Quantitative data was collected without personally identifiable information. For interviews, pseudonyms were used in transcripts and reports, and all identifying details were removed. All collected data was stored securely on password-protected devices, accessible only to the research team.
- Privacy:** Participants' privacy was respected at all times. Interviews were conducted in private settings, and survey responses were collected through secure online platforms to prevent unauthorized access.
- Avoidance of Plagiarism:** All sources of information, whether from literature reviews or direct quotes from participants, were properly acknowledged and cited according to academic standards. Plagiarism detection software was utilized to ensure the originality of the written work.

- Accuracy of Findings:** The research was conducted with utmost rigor and objectivity. Data was analyzed systematically, and findings were reported accurately and truthfully, without manipulation or misrepresentation. Any limitations of the study were openly acknowledged.

- Beneficence and Non-maleficence:** The study was designed to contribute positively to the understanding and improvement of internship management systems like InternLink and student satisfaction, with no foreseeable harm to participants. The potential benefits of the research were clearly communicated.

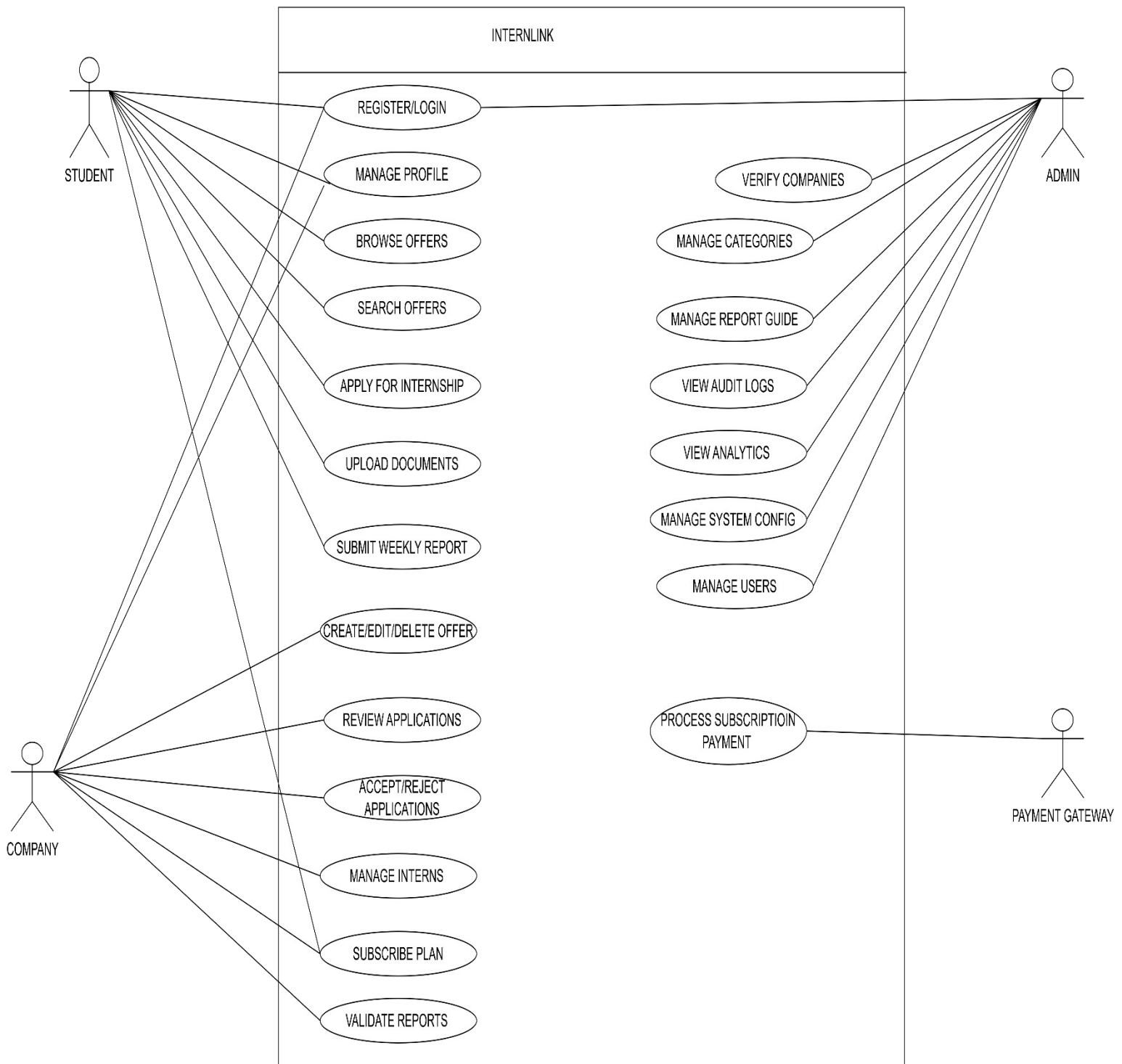
By adhering to these ethical principles, the study aimed to maintain the highest standards of research integrity and ensure that the rights and welfare of all individuals involved were respected.

3.6 STUDY AND PROJECT DURATION

To orchestrate the development lifecycle and establish a predictable project timeline, the **Agile Software Development Life Cycle (SDLC)** model was adopted, supplemented by the **Unified Modeling Language (UML)** standard. UML encapsulates industry-validated engineering best practices essential for abstracting, visualizing, and modeling complex software architectures. While the broader UML ecosystem comprises various diagnostic models including object, deployment, and state transition diagrams this design phase specifically leverages **Use Case**, **Class**, and **Sequence** diagrams to comprehensively articulate Internlink's functional requirements, data schemas, and runtime behaviors.

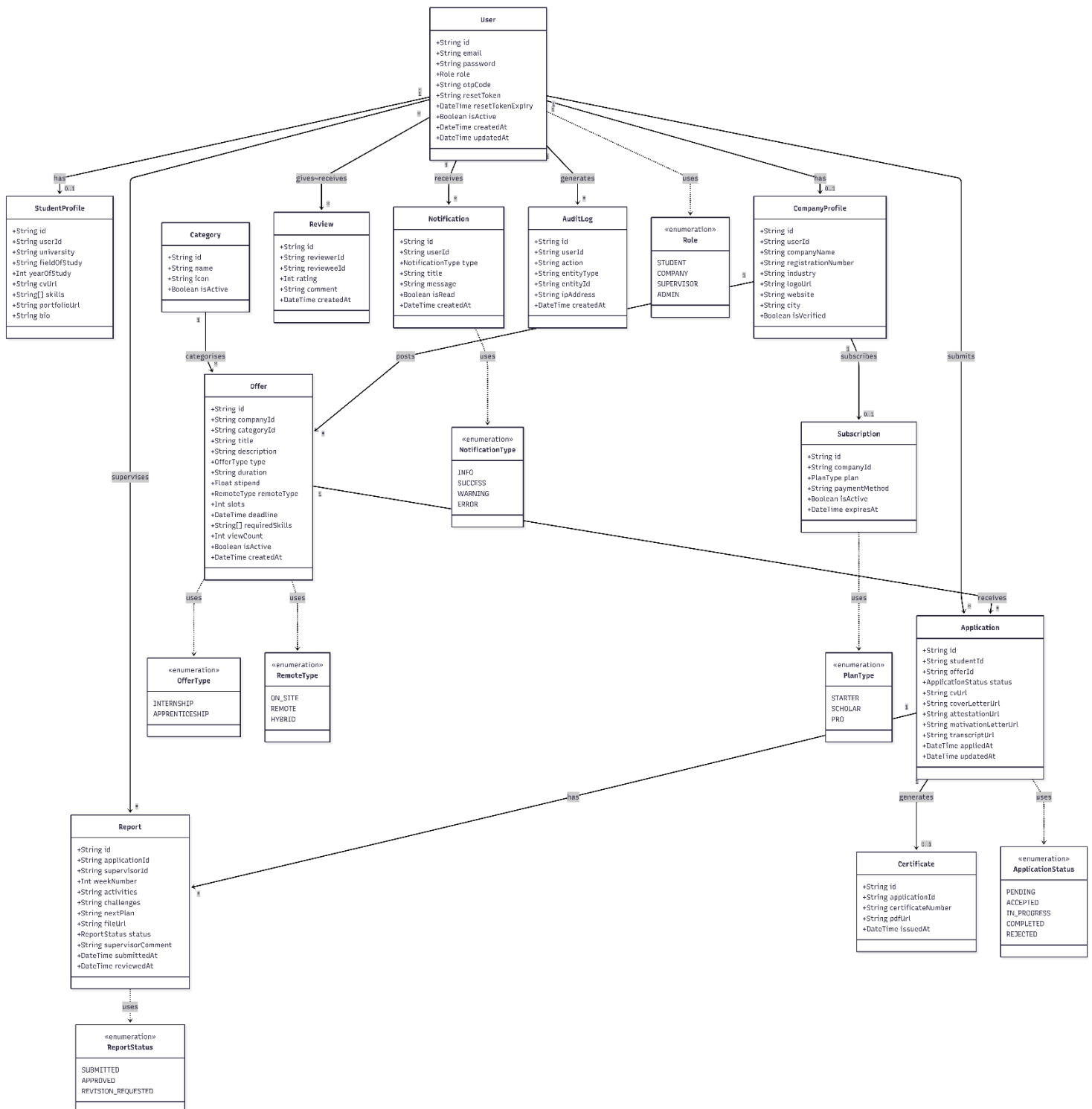
A. USE CASE DIAGRAM

Figure 2: Use case diagram



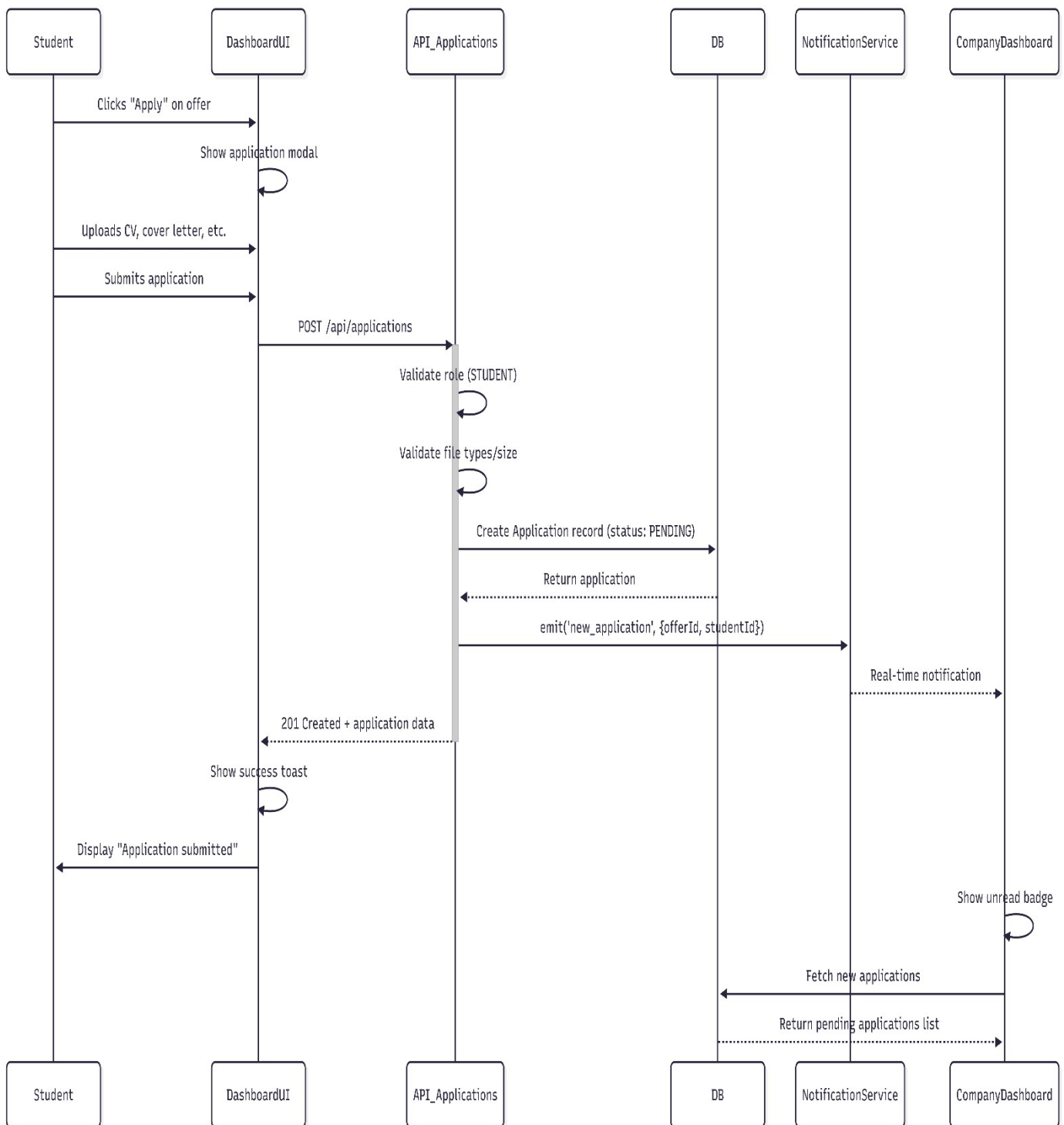
B. CLASS DIAGRAM

Figure 3:Class diagram



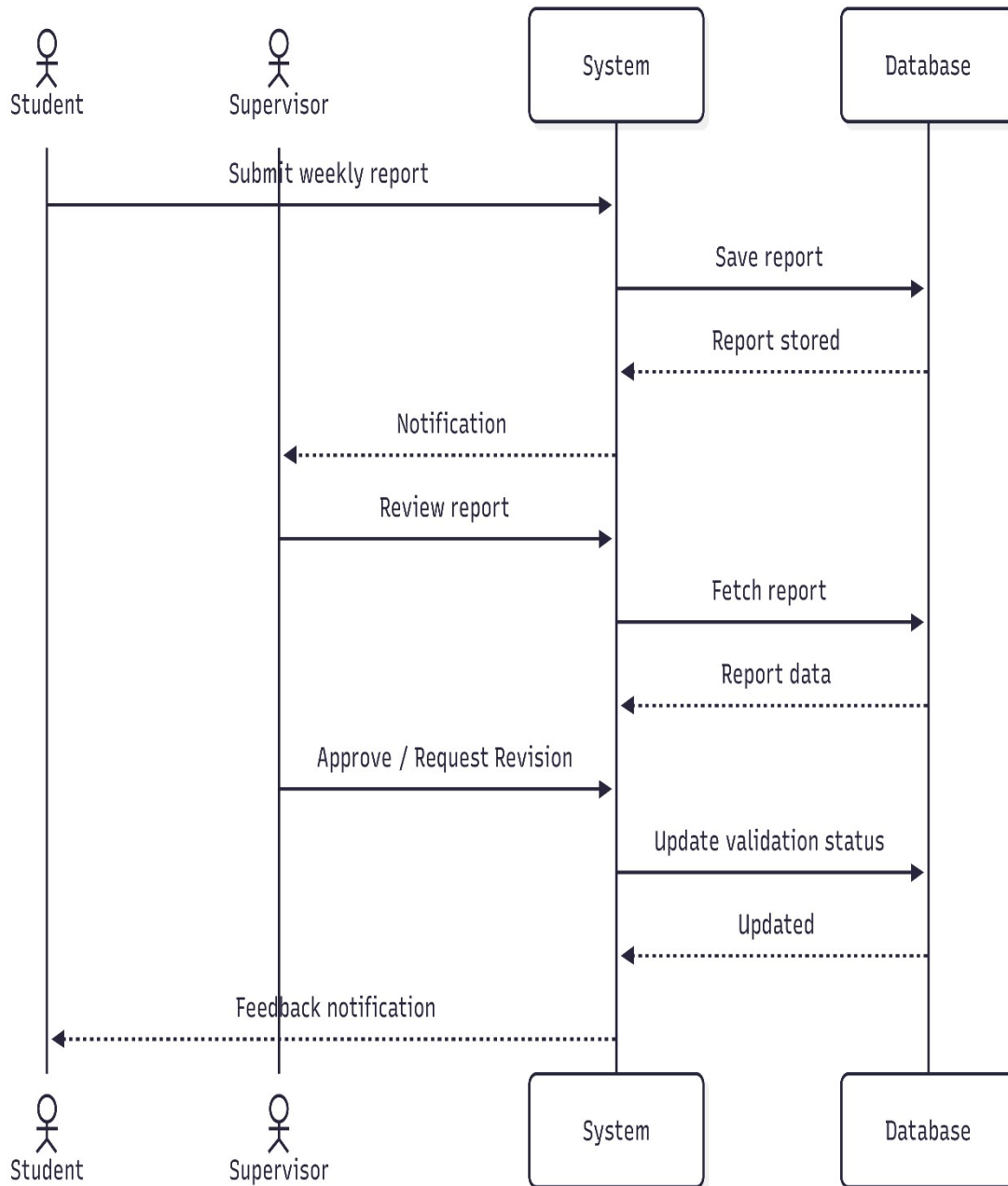
C. SEQUENCE DIAGRAM (STUDENTS APPLY FOR INTERNSHIP)

Figure 4:Sequence diagram (students application)



D. SEQUENCE DIAGRAM (WEEKLY REPPORT)

Figure 5:Sequence diagram (weekly repport)



E. DEPLOYMENT DIAGRAM

Figure 6:Deployment diagram

